

INFO3 Anaïs BELAL

PROJECT REPORT

Numerical Methods Applied to AI

Table of Contents

1. Introduction
2. RL Library Architecture
 - 2.1. Code Organization
 - 2.2. Class Structure
 - 2.3. Technical Choices and Justification
3. Implementation of Reinforcement Learning Algorithms
 - 3.1. ϵ -Greedy
 - 3.2. Upper Confidence Bound (UCB)
 - 3.3. Bandit Gradient
 - 3.4. Policy Iteration
 - 3.5. Value Iteration
 - 3.6. Q-learning
4. Case Studies and Experiments
 - 4.1. SimpleRunner: Movie Recommendation
 - 4.2. YouTubeRunner: Video Feed and Evolving Preferences
 - 4.3. TicTacToeRunner: Learning Through Games
5. Reward Function and Learning
 - 5.1. Reward Modeling
 - 5.2. Adaptation to Environments
6. Conclusion and Remarks

1. Introduction

This project is part of the Numerical Methods for Artificial Intelligence course and aims to design, in Java, a library of Reinforcement Learning (RL) algorithms.

The main objective is to implement different machine-learning strategies allowing an agent to interact with formally described environments while learning to maximize a reward.

The approach is based on formal specifications as the representation of the environment. These are interpreted using ProB, which dynamically generates the state and transition space. From this foundation, we developed a modular library integrating several classical RL algorithms, including ϵ -Greedy, UCB, Bandit Gradient, Value Iteration, Policy Iteration and Q-learning, each tested on different case studies.

This document is the final report and details all the work carried out, from technical choices to experimental results on various environments, ranging from video recommendation (Simple and YouTube) to strategic games such as Tic-Tac-Toe.

2. RL Library Architecture

2.1. Code Organization

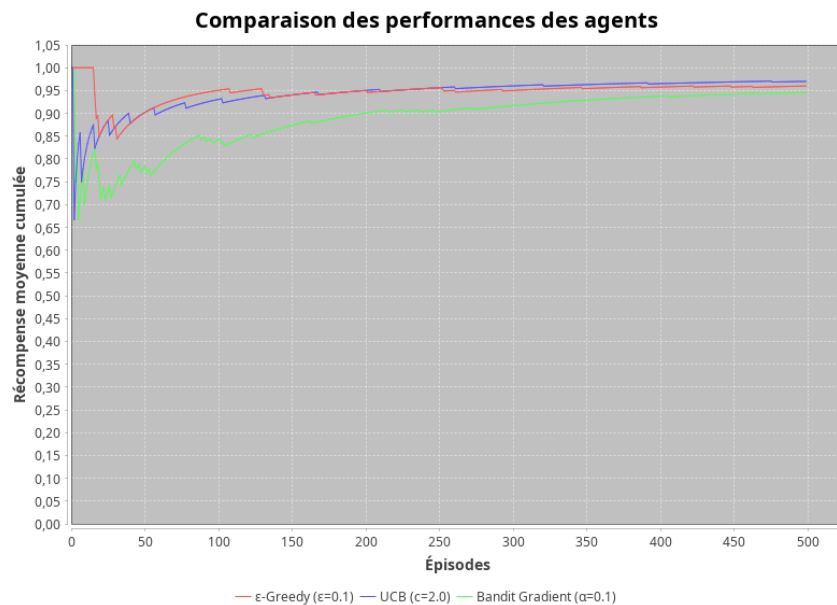
The library is organized as follows:

- Source code is located in `src/main/java/fr/polytech/mnia/`
- The code is divided into several sub-packages:
 - **agents**: contains the different Reinforcement Learning agent implementations.
 - **graphs**: intended for visualization.
 - The remaining Java files (`App.java` and the different runners) are used for testing/demonstration.
- Other directories such as **resources** and **docs** are present for documentation and additional files provided beforehand.

This organization reflects a clear separation of responsibilities: learning agents, visualization, and execution/testing.

2.2. Class Structure

- **Agent.java**: Interface defining the basic behavior of agents. Other classes such as `QlearningAgent.java` or `EpsilonGreedyAgent.java` inherit from it.
- **Specialized agents**:
 - `QlearningAgent`, `PolicyIterationAgent`, `ValueIterationAgent` implement RL algorithms.
 - `BanditGradientAgent`, `UCBAgent`, `EpsilonGreedyAgent` handle multi-armed bandit problems.
- **Visualization.java**: displays agent learning results graphically, including the evolution of action probabilities and cumulative rewards. It also provides tools to compare several agents and visualize their action preferences.
- **App**: main entry point of the application. It runs a test scenario by instantiating one of the Runners defined in the project. To run a scenario, simply choose its number (e.g. 2 for `YoutubeRunner`).



2.2. Class Structure (continued)

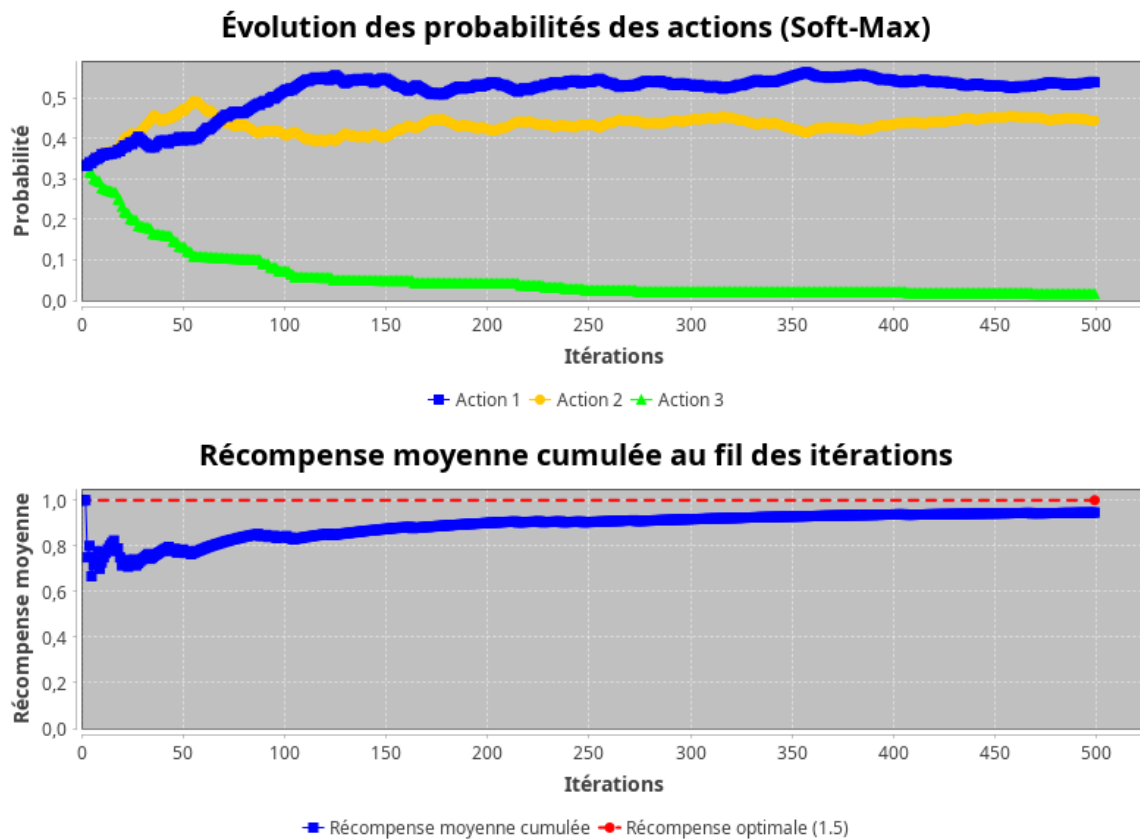
- **Runners:** responsible for executing learning scenarios in different environments (TicTacToeRunner, SimpleRunner and YoutubeRunner). Each class runs the appropriate agents on the same scenario and compares the final results, displaying the appropriate graphs using the imported visualization class. The reward function is defined in the same file.

2.3. Technical Choices and Justification

- **Modularity:** Each agent is in a separate file, improving readability and making modifications easier.
- **Clear naming:** File/class names are explicit (e.g. PolicyIterationAgent.java), making the code easier to understand.
- **Separate runners:** Several classes extending “Runner” make it possible to test different environments without modifying the core agent code.
- **Reward functions:** directly implemented in the runners.
- **Integrated visualization:** A graphical module is included to observe agent behavior.

3. Implementation of Reinforcement Learning Algorithms

The implementation of these agents is explained in the source code through detailed comments. These algorithms are implemented from those studied in class.



4. Case Studies and Experiments

4.1. SimpleRunner: Movie Recommendation

This runner is a comparative test of multi-armed bandit algorithms on a B model specified in `SimpleRL.mch`. It compares three agents: ϵ -Greedy, UCB and Bandit Gradient.

The environment models a user who likes movies A and B, but not C, and each agent must learn this by interacting with the environment for 500 steps. At each step, the agent:

1. Chooses a movie (action) to recommend.
2. Observes a reward (1 if the movie is liked, 0 otherwise).
3. Updates its strategy based on this feedback.
4. Stores the reward for analysis.

Interpretation of results:

ϵ -Greedy

With Q-values $A = 1$, $B = 1$, $C = 0$, the agent chooses almost exclusively movie A (low exploration). It learned that A and B give rewards, but explored B very little. This is the expected behavior with $\epsilon = 0.1$: 90% exploitation, 10% exploration.

UCB

Q-values: $A = 1.0$, $B = 1.0$, $C = 0$. The agent is expected to alternate between A and B, exploring better than ϵ -Greedy. Thanks to the logarithmic exploration bonus, it avoided premature exploitation. It is more balanced in its use of A and B.

Bandit Gradient

Final policy (probabilities): $A = 54\%$, $B = 45\%$, $C = 2\%$ (this varies depending on the test, but the results remain similar). A converged toward a nearly optimal distribution, assigning very little weight to C. Unlike the others, this agent is not deterministic but learns a stochastic policy through Softmax.

--- Résultats finaux pour : ϵ -Greedy ($\epsilon=0.1$) ---				
Vidéo	Valeur	Sélections	Pourcentage	
Tutoriel_Python	0,9820	125	2,50	%
Vlog_de_voyage	0,1992	126	2,52	%
Musique_populaire	0,6115	240	4,80	%
Gaming	1,0000	4509	90,18	%

--- Résultats finaux pour : UCB ($c=2.0$) ---				
Vidéo	Valeur	Sélections	Pourcentage	
Tutoriel_Python	0,9955	2300	46,00	%
Vlog_de_voyage	0,3222	54	1,08	%
Musique_populaire	0,6591	165	3,30	%
Gaming	1,0000	2481	49,62	%

--- Résultats finaux pour : Bandit Gradient ($\alpha=0.1$) ---

Vidéo	Valeur	Sélections	Pourcentage
Tutoriel_Python	0,0124	170	3,40 %
Vlog_de_voyage	0,0016	86	1,72 %
Musique_populaire	0,0027	136	2,72 %
Gaming	0,9833	4608	92,16 %

4.1. SimpleRunner: Graphs

The graph compares the three strategies. ϵ -Greedy converges quickly toward the maximum reward (it exploits the best option very quickly). It is very effective here because the environment is static and simple. UCB learns a little more slowly, but also reaches very good performance. It is useful when a good exploration/exploitation trade-off is desired. Bandit Gradient is slower to converge and remains slightly below the other two. However, it maintains some exploration, especially at the beginning, and achieves good convergence.

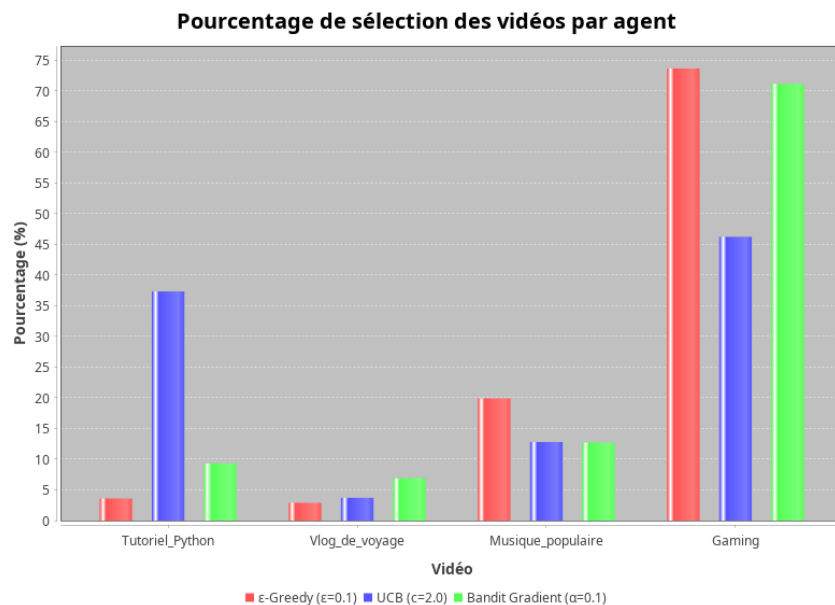
Bandit Gradient: the graph shows the evolution of the selection probabilities of the three movies:

- Action 1 (Movie A) is clearly identified as the best (probability above 0.6).
- Action 2 (Movie B) remains selected with a non-negligible probability (0.35), which is consistent because B also gives a reward.
- Action 3 (Movie C) is progressively abandoned (probability close to 0), because it provides no reward.

This shows that the agent's Softmax policy is correctly learned.

The graph shows the evolution of the cumulative average reward for the Bandit Gradient agent ($\alpha = 0.1$). The blue curve gradually converges toward the maximum reward of 1. This means that the agent effectively learns to favor profitable arms (movies A and B), even though learning is somewhat slower than with other strategies.

In conclusion, all agents achieve good performance, but in this static and deterministic case, ϵ -Greedy and UCB converge fastest. Bandit Gradient, although somewhat slower, learns to favor good movies while maintaining an adaptive policy. This is exactly what is expected from an intelligent recommendation system in a static environment.



4.2. YoutubeRunner: Evolving YouTube Video Recommendation System

The YoutubeRunner class simulates a YouTube video recommendation system using the same agents. It relies on a non-deterministic B model (YouTube.mch) where the reward is proportional to the viewing duration of a video, thus reflecting user interest. The runner records the rewards obtained and the selection frequency of each video, allowing a comparative analysis of learning strategies under dynamic user preferences.

Interpretation of results:

These results correspond to one test run; the conclusions remain similar.

ϵ -Greedy ($\epsilon = 0.1$)

This agent favors exploitation of the most profitable known option, with limited exploration. It quickly identified “Gaming” as the best option, which explains its majority selection. Similarly, it often chooses popular music. However, it does not exploit other options that could eventually be interesting.

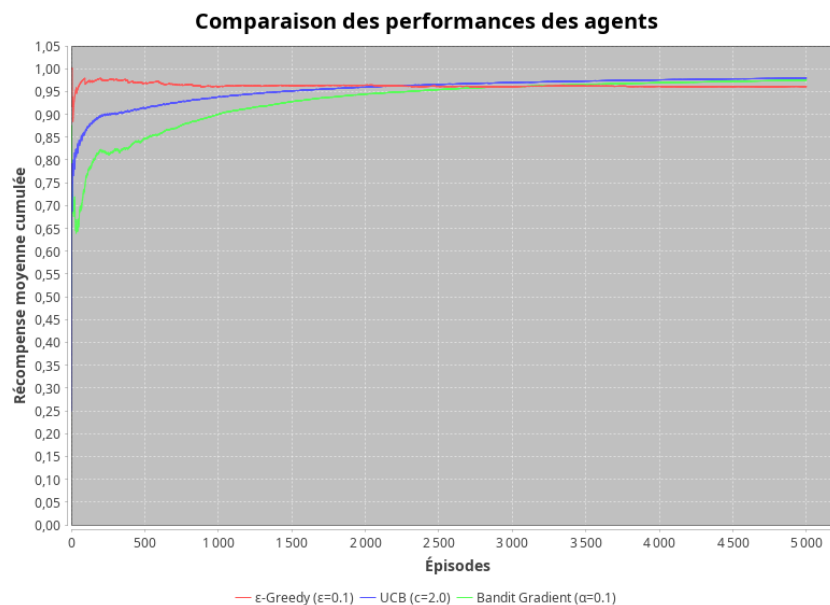
UCB agent ($c = 2.0$)

The UCB agent balances exploration and exploitation by taking uncertainty in the estimates into account. It adapted to changes in preferences, particularly by increasing the selection of “Tutoriel_Python” when its interest increased. This ability to adapt makes it effective in dynamic environments.

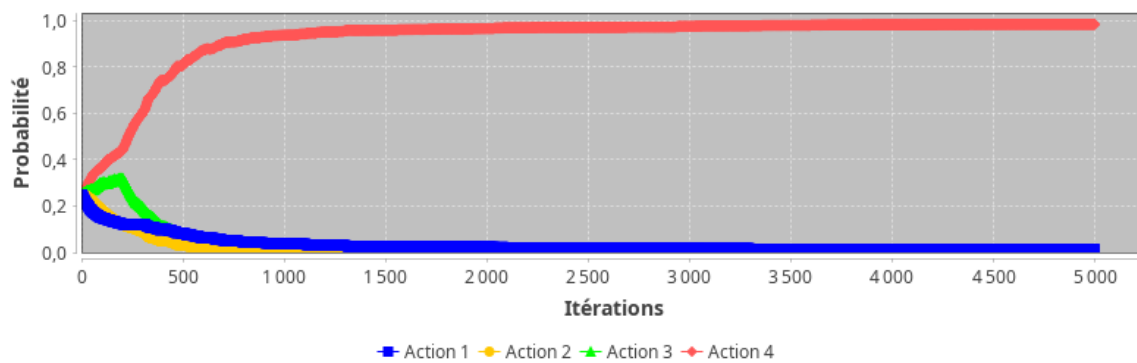
Bandit Gradient agent ($\alpha = 0.1$)

This agent adjusts its preferences according to the rewards obtained, without explicitly estimating action values. It strongly favored “Gaming”, but showed limited adaptation to changing preferences, first toward “Musiques populaires” and then toward “Python_tutoriels”.

In conclusion, among the three agents, UCB and Bandit Gradient showed the best ability to adapt to changing user preferences, effectively balancing exploration and exploitation. ϵ -Greedy, although effective in stationary environments, can be less effective when preferences evolve.



Évolution des probabilités des actions (Soft-Max)



4.2. YoutubeRunner: Graphs

This graph shows the percentage of selection of each video by the three agents after 1000 iterations.

- Gaming is the most attractive video for all agents, indicating that it offers the highest average reward (high viewing time).
- ϵ -Greedy strongly exploits the best video, with little exploration.
- UCB maintains good exploration, especially for Tutoriel_Python, which seems to become better over time.
- Bandit Gradient shows good adaptation, concentrating probability on Gaming. Initially, it focuses on popular music before switching to exploring Python tutorials, which seem increasingly interesting.

ϵ -Greedy performs very well here because the environment provides a stable reward for Gaming, which it learns to exploit quickly. UCB explores more, so it takes longer to converge but remains robust. Bandit Gradient adapts slowly, which may be due to the changing dynamics of user preferences.

Bandit Gradient: the graph shows how video selection probabilities evolve over time (blue: Tutoriel_Python; yellow: Vlog_de_voyage; green: Musique_Populaire; red: Gaming).

- Action 4 (red, Gaming) becomes dominant (> 85% at the end).
- Action 3 (green, music) is chosen at the beginning but is quickly abandoned.
- Travel vlogs are very little explored from the beginning.
- The agent explores Python tutorials more than popular music, adapting to new preferences, but eventually abandons this option as well, always preferring Gaming.

4.3. TicTacToeRunner: Learning Through Games

This runner tests different reinforcement learning agents in a deterministic environment: Tic-Tac-Toe. The agent learns to play against a random opponent (Player 0 in our implementation) through `TicTacToe.mch`, implemented in ProB. Rewards are: +1 for a win, 0 for a draw or unfinished game, and -1 for a loss.

We implemented and tested three approaches: Policy Iteration, Value Iteration and Q-learning.

The comparison is performed in `TicTacToeRunner`, in the `compareAgents()` method, which launches the learning of the three agents, compares them using their accumulated average rewards, and launches the interactive version of Tic-Tac-Toe against the agent with the best performance (the player is Player 0, the agent is Player 1).

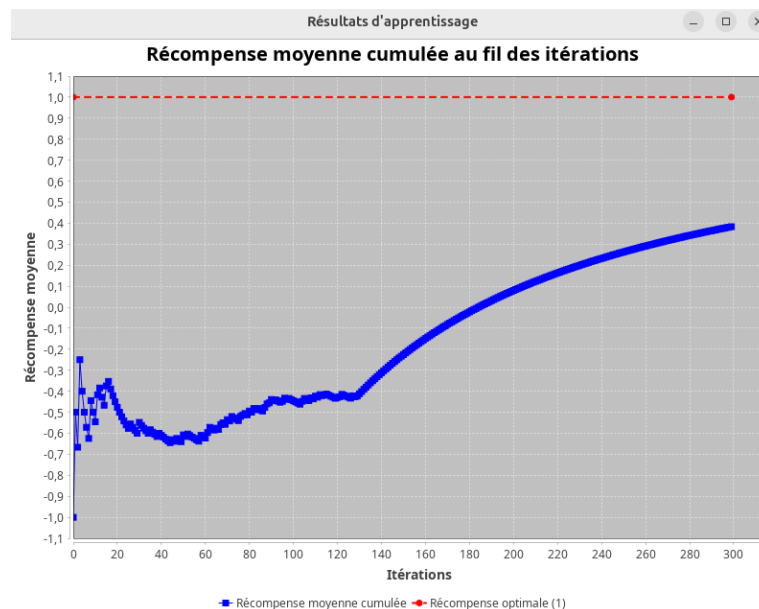
For each agent we used the following standard parameters: $\epsilon = 0.1$, $\gamma = 0.9$, $\alpha = 0.1$ (see `App.java` to change them).

Note: For Tic-Tac-Toe, we implemented a new `initialiseAleaInitialState()` function in `Runner` to obtain different initial states from one episode to another, making exploration easier at the beginning of learning.

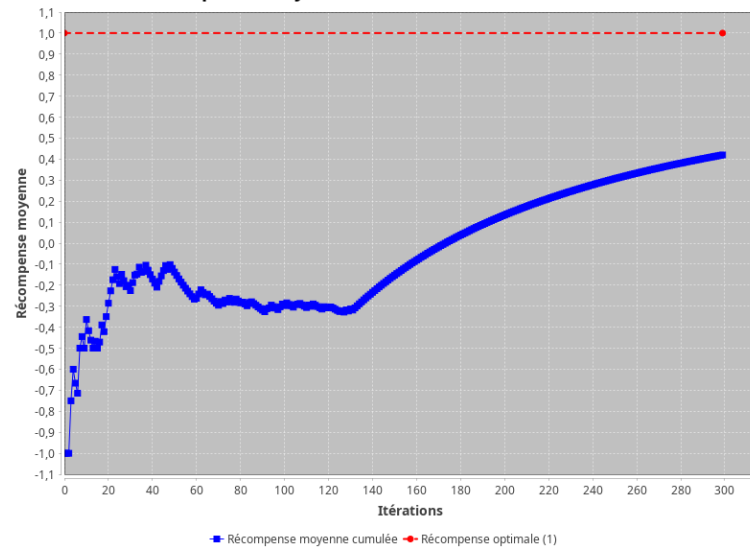
Policy Iteration:

In our Policy Iteration implementation, we chose to have a long exploration phase followed by exploitation, because the Tic-Tac-Toe environment is stable and we needed many transitions and visited states before starting learning. Since there are approximately 19,000 possible states, we chose to start exploitation after 2,000 visited states/transitions (also adjusted according to the number of episodes played, because with too few episodes there is no exploitation due to insufficient data).

Note: The execution of `PolicyIterationAgent` during the learning phase is very slow. We could probably have improved computational efficiency by sorting the list of visited states and the associated transitions and rewards beforehand (using a system somewhat similar to memory caches), but we did not have time.



Récompense moyenne cumulée au fil des itérations



PolicyIterationAgent over 300 episodes

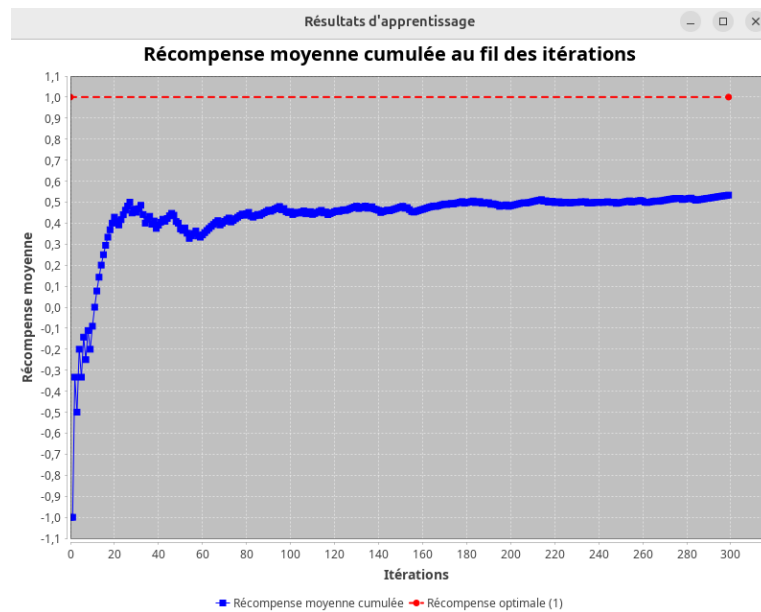
Here we can clearly see the difference between the exploration phase (around 130 episodes in this case) and the exploitation phase, where the agent almost never loses.

ValueIteration:

In our ValueIteration implementation, as with PolicyIteration, we chose a long exploration phase followed by exploitation, for the same reasons mentioned above: a stable environment and many possible states. Unlike PolicyIteration, ValueIteration runs faster because the calculations are less costly.

ValueIterationAgent over 300 episodes

We again clearly see the separation between exploration and exploitation. Around 130 episodes, we see the transition after which the agent almost never loses. This similarity is explained by the fact that we used the same experience threshold as for PolicyIteration, namely 2,000 visited states.



Q-learning:

The Q-learning agent was simpler to implement. Updating the Q-table was faster than maintaining the lists of states and transitions required by model-based algorithms such as Policy Iteration and Value Iteration. Q-learning is a model-free algorithm, which also results in faster convergence.

Here we observe much faster convergence than with the two previous agents. However, it seems to stagnate around a reward of 0.5, while the other two seem to tend toward a higher value, which is consistent with the theory. (See the end of section 4.3 for a series of 500 episodes per agent.)

- Q-learning converges quickly toward 0.5, which is consistent because it is model-free, making it more interesting when the agent is trained for a relatively small number of episodes (here 300).
- Policy Iteration explores for some time before converging toward 0.39, but with more training its performance seems to surpass that of Q-learning.
- Similarly, Value Iteration explores at the beginning and then exploits, quickly tending toward 0.41.

Note: In theory, Policy Iteration, even though its calculations are more expensive, should converge faster than Value Iteration. We may have needed to start the Policy Iteration exploitation phase earlier to obtain better results instead of using the same threshold as Value Iteration.

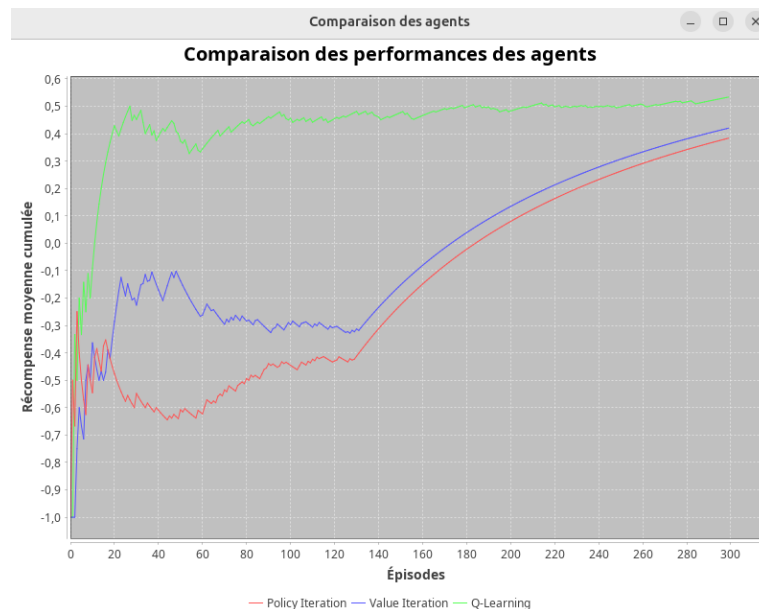
Interactive part: Player versus trained agent

We observe that the agent does not really manage to counter the player. Several reasons are possible:

- Lack of training
- The agent was trained only against a random player, which was not sufficient
- An error in the application of the algorithm

Curves for 500 episodes per agent

As expected, the greater the number of episodes, the more QlearningAgent is overtaken by Policy IterationAgent and Value IterationAgent, which evolve in a similar way.



5. Reward Function and Learning

5.1. Reward Modeling

➤ In SimpleRunner:

The SimpleRL.mch model is deterministic. The user likes movies A and B, but not C. The reward is therefore modeled simply in the code as:

```
// String res = getState().eval("res").toString();  
// double reward = res.equals("OK") ? 1.0 : 0.0;
```

If the user likes the movie, res = OK, so reward = 1. Otherwise, reward = 0.

➤ In YoutubeRunner:

The YouTube.mch model is non-deterministic and more realistic. The reward depends on the viewing duration of a video:

```
// String rawDuration = getState().eval("duration").toString();  
// double reward = parseDuration(rawDuration); // e.g. "25" becomes 0.25
```

The parsing function transforms a string representing a duration (such as "25" or "duration = 100") into a normalized numerical value between 0.0 and 1.0, usable as a reward in a multi-armed bandit algorithm; it returns duration / 100.0.

Here, the reward is a continuous value between 0 and 1, representing user engagement.

➤ In TicTacToeRunner:

The TicTacToe.mch model is deterministic and there are only three outcomes:

- Win: +1
- Draw or game in progress: 0
- Loss: -1

The method `getState().eval("win(1)").toString()` is used to evaluate a win by Player 1 (or Player 0).

```
>>> Meilleur agent : Q-Learning avec moyenne 0.5333333333333333  
== Partie interactive contre l'agent entraîné ==  
1 1|1 2|1 3  
-----  
2 1|2 2|2 3  
-----  
3 1|3 2|3 3  
C'est votre tour. Entrez les coordonnées x y (1 à 3) :  
2 2  
  | | |  
  | 0 |  
  | | |  
-----  
Coup de l'Agent  
1 | | |  
-----  
  | 0 |  
  | | |  
-----  
C'est votre tour. Entrez les coordonnées x y (1 à 3) :  
2 1  
  | | |  
  | 0 |  
  | 0 |  
  | | |  
-----  
Coup de l'Agent  
1 | | |  
-----  
  | 0 |  
  | | |  
-----  
  | | |  
-----  
C'est votre tour. Entrez les coordonnées x y (1 à 3) :  
2 3  
  | | |  
  | 0 | 0  
  | | |  
-----  
  | | |  
-----  
Vous avez gagné !  
Voulez-vous rejouer ? (o/n) o
```

5.2. Adaptation to Environments

➤ In SimpleRunner:

The environment is fixed: the user does not change their mind. The agents learn stable preferences because A and B are expected to be selected increasingly often and C less and less. All algorithms converge relatively quickly.

➤ In YoutubeRunner:

The environment is dynamic: user preferences change over time. The algorithms must therefore adapt their policy to these variations. For example, an agent must gradually abandon music in favor of Python tutorials.

➤ In TicTacToeRunner:

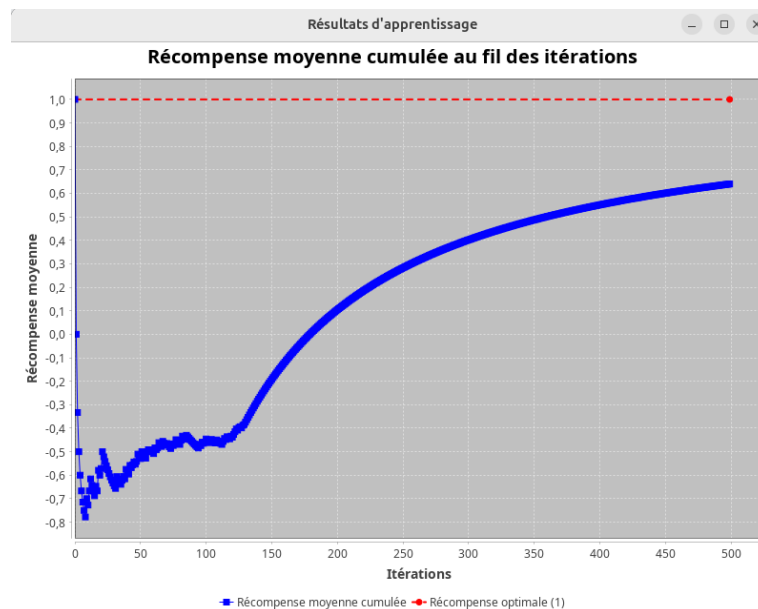
The environment is fixed, the possible states are finite, and the winning conditions do not change. The environment can therefore be considered stable, which is why we chose a long exploration phase in Policy Iteration and Value Iteration, followed by an exploitation phase.

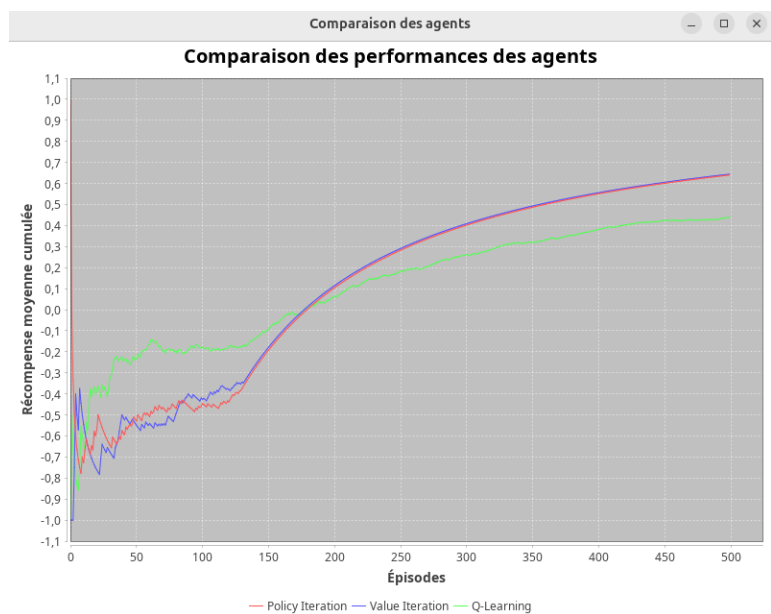
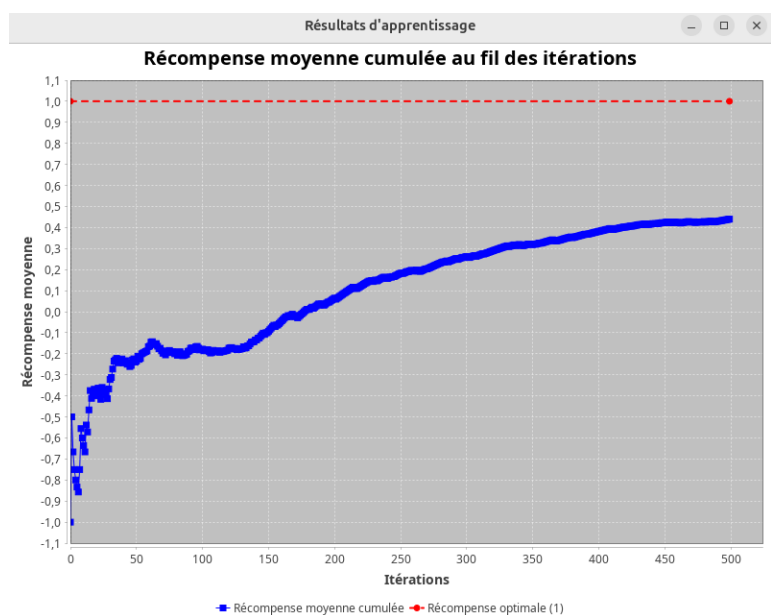
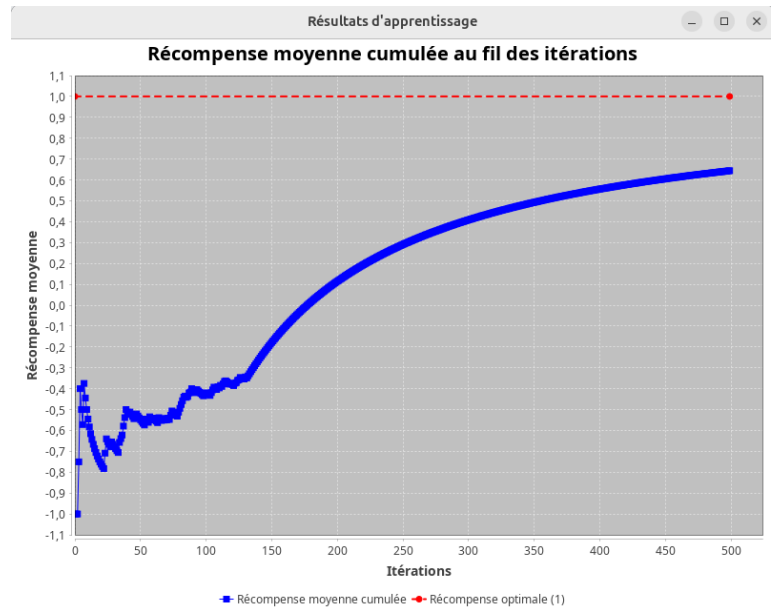
6. Conclusion and Remarks

This project allowed us to design a versatile reinforcement learning agent library capable of adapting to several types of environments. By addressing both simple and more complex cases such as Tic-Tac-Toe and personalized recommendation, we were able to experiment with the advantages and limitations of each approach.

Iterative agents (Value Iteration, Policy Iteration) proved very effective when the environment was fully known and stationary, while exploratory methods such as Q-learning and multi-armed bandits responded better to partially observable environments or dynamic contexts.

Beyond the technical implementation, this project helped us better understand the subtleties of the exploration/exploitation trade-off and the importance of choosing an algorithm according to the characteristics of the situation.





Remarks

We chose to use ChatGPT for the implementation of the `Visualization.java` class. Since this project required a significant investment in algorithmic logic and modeling, we considered that coding the graphs was not a central pedagogical skill expected in this context.

The main objective was to test a library of reinforcement learning algorithms, so we considered it more relevant to focus on this aspect.

In addition, we had to modify the B machine in the `YouTube.mch` file because, in its initial version, the variable "duration" was not directly accessible through `eval("duration")` in the Java code. In the original model, "duration" was returned as the output value of the choose operation (`duration <- choose(vv)`), but it was not defined as a global state variable.

To work around this limitation and allow simplified access from our Java code, we:

- Added "duration" as a state variable in the VARIABLES section.
- Modified the choose operation so that it no longer returns a value, but directly modifies the "duration" variable on each call.

Thanks to this modification, we were able to use the line `getState().eval("duration")` to retrieve the viewing duration after each video selection, considerably simplifying integration with our agents.